

How LLM reads your Python?

From Transformer Circuits Perspective to Hands-On Tracing of Open-Source Models

Anna Liang, PyCon TW 2026



theory/attention/

Attention is a kernel average.
 $\text{softmax}(qk^T/\sqrt{d})V$ is Nadaraya and Watson (1964)
with the kernel $e^{-q \cdot k/\sqrt{d}}$.
Kernel = **QK** (*where*).
Values = **OV** (*what*).

How to think 2020 to 2022: the frame

2020 Zoom In
features, circuits, universality

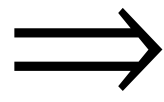
2022 Induction heads
[A] [B]...[A] → [B]

Why “circuit”? Neuroscience and electronics: the function is in the wiring, and the wiring is written down. Narrow sense: a head’s QK / OV path. Broad sense: a prompt’s attribution graph.

2021 Framework
head = QK (where) × OV (what)

2022 Superposition
more concepts than dimensions

the hinge
more features
than
neurons ⇒ you
need
a learned
dictionary



How to look 2023 to 2025: the instruments

2023 SAE $x \approx W_{\text{dec}}f(x)$
sparse, wide: “what is here”

2024 Scaling Mono, Gemma Scope
code-error feature; open dictionaries

2024 Transcoder $\text{MLP}(x) \approx W_{\text{dec}}f(x)$
“what this layer computed”; edge = $a_i(d_i \cdot J \cdot e_j)$

2025 Circuit Tracing
attribution graphs; circuit-tracer, Neuronpedia



theory/circuits/
Why “circuit”?

1 The question

Cut a Python function off right after `return`. The model must choose *which local variable to return*:

P1 return-variable 'result' 95.7, 'ans' 0.0

```
def calculate_total(prices):  
    result = 0  
    for price in prices:  
        result = result + price  
    return
```

Binding, or just the name on the page?

2 Four prompts

P2 and P3 pull the two apart; both are **deliberately invalid Python** (a name is read before assignment). P4 is valid.

P2 split-binding 'result' 68.8, 'ans' 25.2

```
def calculate_total(prices):  
    ans = 0  
    for price in prices:  
        result = result + price  
    return
```

P3 result-never-bound 'ans' 89.1, 'result' 6.5

```
def calculate_total(prices):  
    ans = 0  
    for price in prices:  
        ans = result + price  
    return
```

P4 two-valid-locals, valid 'result' 82.4, 'ans' 11.2

```
def calculate_total(prices):  
    ans = 0  
    result = 0  
    for price in prices:  
        result = result + price  
    return
```

P3 settles it: present but never bound loses, 89.1 to 6.5. Not about the word `result` either: 18 names × 4 patterns, top-1 over the vocabulary.

pattern	valid?	top-1 ok	mean margin
consistent (A bound twice)	yes	18/18	+94.8 pp
two-valid (P4 shape)	yes	18/18	+59.6 pp
never-bound (A only read)	control	18/18	+89.0 pp
split (D line 2, A in loop)	control	17/18	+26.5 pp

The one miss (score, split) is an exact 44.92 / 44.92 tie, counted as a miss.

3 Four reading traps

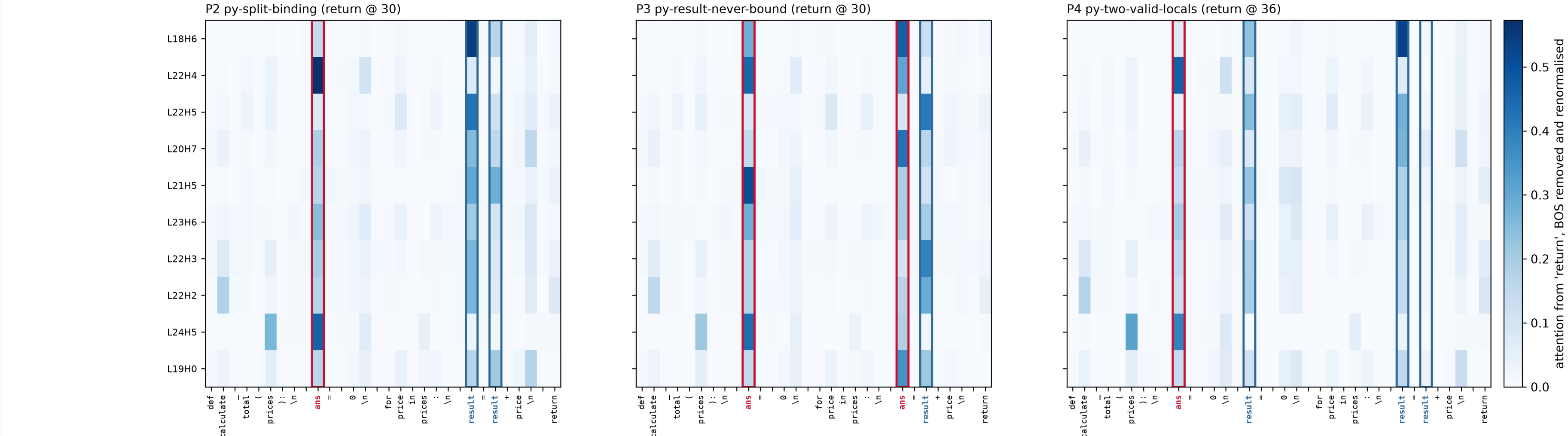
- **A direct effect means nothing until you name the logit.** 18_4769 in P3 is +0.028 to 'ans' and **+8.50** to 'result': same node, opposite story. Every number here names its target.
- **The pinned subset can invert the answer.** P3's pinned nodes say `result` +12.8 vs `ans` +11.9, opposite to the 89 / 6 outcome. Summing *every* node gives +26.96 vs +17.03. Put the totals beside every subgraph.
- **Amplifying misleads.** 18_4732 scaled up 8× *decreases* P('ans') to 0.1%. Scale down, never up.
- **Two agreeing numbers are not a law.** The winner's total is 26.96 / 26.96 in P2 / P3 and 37.30 / 38.88 / 28.04 in the other graphs. The ordering holds 5 / 5; the magnitude is an observation.

4 Limitations

- **One model**, because the method needs a full public transcoder set; **no cross-model claim**. google/gemma-2-2b, GemmaScope transcoders (7.4 GB; Llama-3.2-1B 25.4, Qwen3-1.7B 71.2, GPT-OSS 20B 264 GB), one RTX 3090, bfloat16 (never float16: gemma-2 overflows it).
- **Coverage is uneven:** `ans`'s drive is embedding-led and its suppressor is not captured.
- **Single-token cuts** probe recognition, not generation. Magnitudes depend on pruning thresholds; orderings and near-zero cross terms do not.

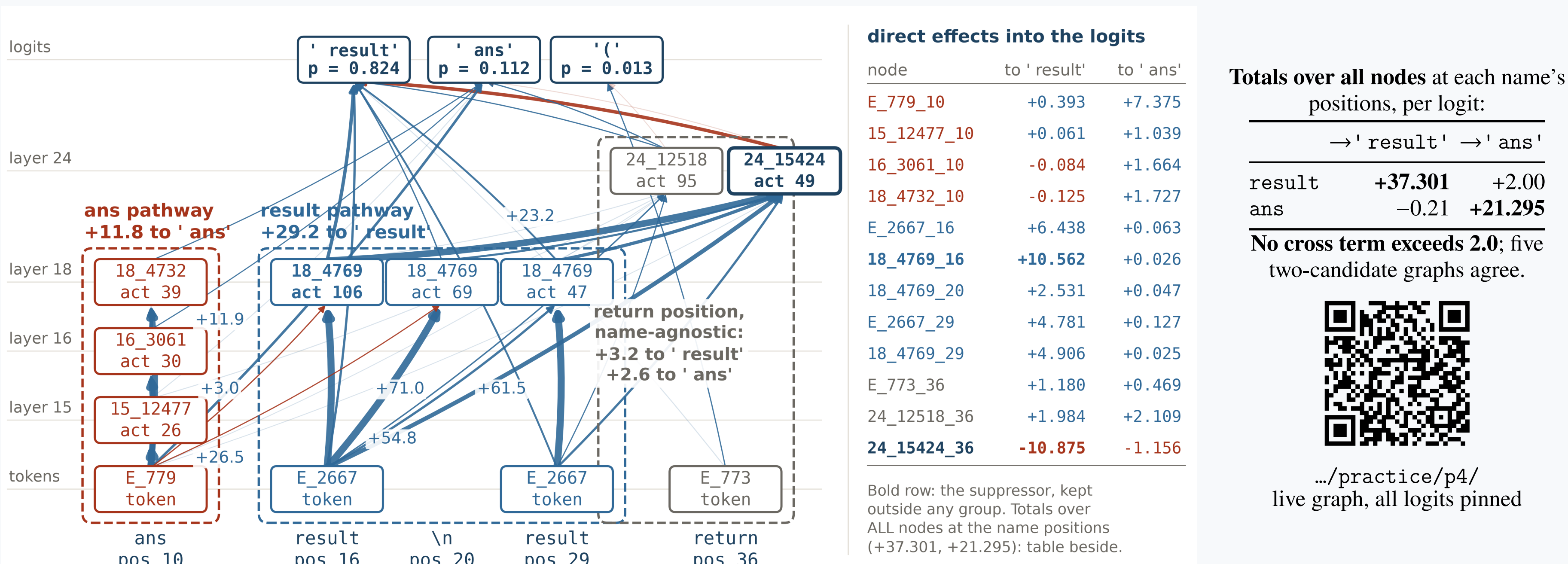
5 WHERE: what return looks at

Where the 'return' position looks: the 10 heads with most mass on result/ans in P4 (rows fixed across panels). Blue boxes: result tokens, Red boxes: ans tokens.



Attention from `return`, BOS removed and renormalised; rows are the 10 heads with most mass on `result` / `ans` in P4, the same rows in all panels (blue `result`, red `ans`). **Attention says where, not what; importance is verified by steering** (panel 7). The heads look at the same *places* in P2 and P3 (median cosine 0.86; the assignment line and the loop line); what changes is which name sits there. L18H6 and L21H5 are among the strongest induction heads (0.79, 0.67 on repeated random sequences): the 2022 cell above.

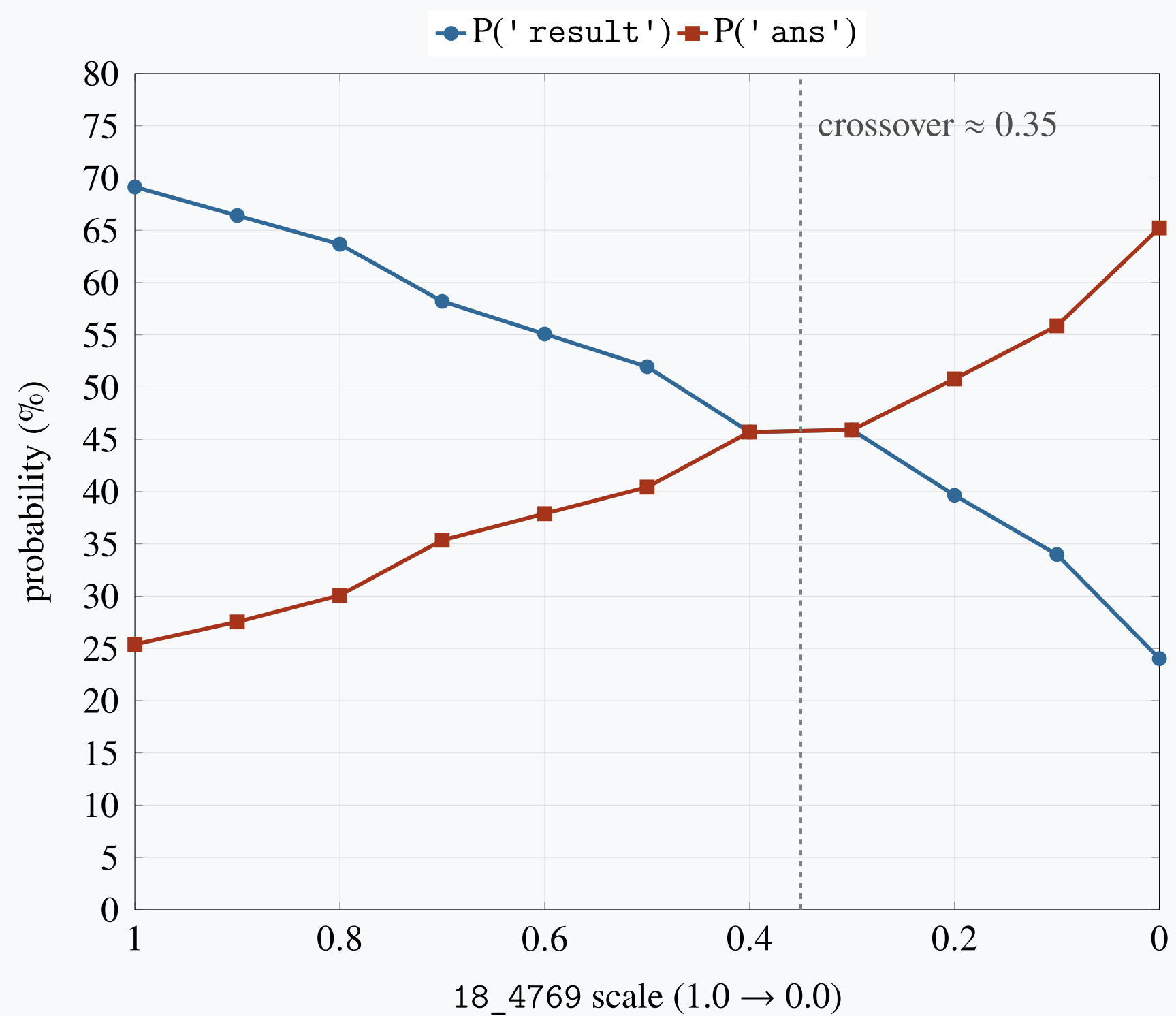
6 WHAT: the P4 attribution graph



Pinned subgraph of the P4 attribution graph (node threshold 0.7), drawn from the graph file: edge width is |weight|, blue positive, red negative; the table lists each pinned node's direct effect against a named logit. One pathway per name, and they barely touch. 18_4769 carries result (activation 103 on P1's first result; silent on `ans` = 0, so tied to the *name*). At return, 18_4769_16 gives **+10.562** to 'result' and 24_15424_36 opposes it at **-10.875**, both against 'result': the two largest nodes in the project, nearly cancelling. Read the pinned subset with the totals: binding is not a gate, it sets how much drive a name accumulates, then the logit layer takes the winner.

7 DOES IT MATTER

Scaling 18_4769 down in P2, every step inside its natural range:



Monotonic, crossing at ≈0.35 (0.4 / 0.3 tie from bf16 quantisation). **Turn one pathway down and the model returns the other variable.** Ablations, an independent method, agree:

condition	P('result')	P('ans')	argmax
P2 baseline	69.14	25.39	'result'
P2 ablate 18_4769	24.02	65.23	'ans', flipped
P2 ablate 18_4732	73.44	21.00	'result'
P1 ablate 18_4769	95.31→69.92	0.00	'result'
P3 ablate 18_4769	6.45→0.93	94.53	'ans'
P3 ablate 18_4732	8.06	89.06→86.72	'ans'

Last row: removing `ans`'s feature barely moves 'ans', exactly as attribution predicted (its drive sits in embeddings). Two independent methods, one answer. The baseline forward pass reproduces the behavioural probe (95.31 / 69.14 / 6.45) first.

8 AI wrote this? Look twice.

It represents *what a name is bound to*, not *when* or *in which scope*. Ground truth: each snippet is executed.

Generator-expression late binding (PEP 289). The iterable is captured at creation; the filter sees the rebound name:

```
>>> lst = [1, 2, 3, 4, 5]  
>>> g = (n for n in lst if n in lst)  
>>> lst = [0, 1, 2]  
>>> print(list(g))  
[
```

Correct next token 1 (the old list is iterated); model: 0 at **55.1%**. Desugared as PEP 289 defines it: a 40.4 / 40.4 tie. The eager-vs-lazy cut gets **0.00%**.

Ten binding traps, scored in pairs. Same surface, opposite answers:

```
>>> for i in range(3):  
...     pass  
...  
>>> print(i)  
NameError: a comprehension has its own scope.
```

prints 2: the loop variable leaks.

Model: Traceback at 35.6%.

Both rules, swapped. Per case 4 / 10; **as pairs, 0 / 4:**

pair	what the model does
lambda: i vs lambda i=i: i	[0, 1, 2] to both
acc=[] vs acc=None	[1, 2] to both
loop-var leak: for vs comprehension	exactly inverted
class-body comprehension, illegal vs legal	legal ok; illegal at noise

Conclusion

In gemma-2-2b, which local gets returned is decided by how much drive each name's *own* pathway accumulates, not by a binding gate: the two pathways barely touch, and turning one down flips the answer. Attention says *where* (the same places in every prompt), the graph says *what*, steering says *it matters*. **Next:** split the +10.562 edge by attention head, tying the heatmap to the graph; a second model (Llama-3.2-1B); cuts longer than one token.